

Universidad de los Andes

FACULTAD DE INGENIERÍA Y CIENCIAS APLICADAS

Métodos Estadísticos para la Gestión

Proyecto 2 — Informe 2

Integrantes

Agustín Álvarez

Arantxa Toledo

Samuel Vallete

Profesor

Jorge Gomez Miranda

Informe 2 — Assembly documentado, medición de tiempo y promedio de diez rondas

Introducción

Este informe documenta el programa sw/game.s, implementado en ensamblador RV32E para el Espino Core, que constituye el juego de reflejos propiamente dicho sobre el Pochoco SoC. Este documento se concentra en la lógica de software: cómo se estructura cada ronda del juego, cómo se mide el tiempo de reacción del jugador, cómo se calcula y muestra el promedio de las diez rondas correctas, y cómo se maneja una pulsación incorrecta.

El programa está escrito íntegramente para una ALU sin soporte de shifts ni de multiplicación o división nativas, por lo que buena parte de la lógica aquí documentada consiste en reemplazar esas operaciones con sumas y restas repetidas. Este informe explica, paso a paso, cómo se resuelven estas limitaciones dentro del flujo del juego.

Se cubren los siguientes aspectos: el mapa de registros y las convenciones usadas a lo largo del programa, el flujo general de una ronda completa, el reaprovechamiento del mecanismo de espera de 3 segundos descrito en el Informe 1, la medición del tiempo de reacción y su conversión a décimas de segundo, la conversión del resultado a BCD para los displays de 7 segmentos, el manejo de pulsaciones incorrectas, y, finalmente, la acumulación y el cálculo del promedio de las diez rondas correctas, incluyendo el criterio de redondeo aplicado.

Mapa de registros y convenciones usadas

El programa documenta explícitamente, en la cabecera de game.s, el rol de cada registro usado. Dado que el Espino Core implementa RV32E (16 registros, x0–x15), el programa completo —incluyendo las 10 rondas, la medición de tiempo, la conversión a BCD y el manejo de errores— se resuelve dentro de ese espacio reducido de registros, sin necesidad de guardar/restaurar valores en memoria (no hay pila ni llamadas a subrutinas con retorno).

Registro	**Rol**
x0	Constante cero (fija en RV32E/RISC-V)
x1	Dirección base de periféricos (0x8000_0000)

Registro	Rol
x2	Máscara one-hot del LED objetivo de la ronda actual
x3	Último valor leído del registro de botones
x4	Valor armado para escribir en los displays (en BCD de 2 dígitos)
x5	Registro de uso general / scratch (cálculos intermedios)
x6	Contador de rondas correctas acumuladas (0 a 10)
x7	Suma acumulada de las décimas de segundo de las 10 rondas
x8	Décimas de segundo de la ronda actual
x9	Marca de tiempo inicial (t0) de la fase en curso
x10	Marca de tiempo final (t1) / tiempo transcurrido (elapsed)
x11	Contador auxiliar (usado en las conversiones a BCD y en la división por 10)
x12	Constante TENTH (2.500.000 ciclos = 1 décima de segundo)
x13	Constante TRES_S (75.000.000 ciclos = 3

Registro	Rol
	segundos)
x14	Constante DIEZ (valor 10, usado en las conversiones decimales)
x15	Registro de uso general / scratch (cálculos intermedios, p. ej. umbral de redondeo)

Todos los accesos a periféricos parten de la dirección base cargada en x1 (0x8000_0000) y se expresan como offsets fijos sobre esa base, definidos como constantes .equ al inicio del programa:

Constante	Offset	Periférico
DIGITOS	0x00	Displays de 7 segmentos (escritura)
LEDS	0x04	LEDs (escritura)
BOTONES	0x08	Botones (lectura)
CICLOS	0x0C	Contador de ciclos de 32 bits (lectura, ver Informe 1)

Esta convención permite que cada acceso a periféricos en el código se exprese de forma legible, por ejemplo sw x2, LEDS(x1) o lw x3, BOTONES(x1), en lugar de usar offsets numéricos directamente.

Además de las direcciones, el programa define tres constantes que se cargan una sola vez en registros al inicio (_start) y se mantienen fijas durante toda la ejecución: TENTH (x12) y TRES_S (x13), justificadas numéricamente en el Informe 1, y DIEZ (x14), usada tanto para las conversiones a BCD como para detectar que se completaron las 10 rondas del juego.

Flujo general del programa

El programa se organiza en torno a una etiqueta central (ronda:) que se ejecuta una vez por cada intento del jugador y a la que el programa vuelve a saltar tanto tras un acierto (si aún no se completan las 10 rondas) como tras un error (reiniciando solo la ronda actual). El flujo completo es el siguiente:

Inicialización (_start)

Antes de entrar al bucle de rondas, el programa carga la dirección base de periféricos (x1), las constantes de temporización y conteo (x12–x14), inicializa en 0 el contador de rondas correctas (x6) y la suma acumulada de décimas (x7), y apaga displays y LEDs para partir en un estado conocido.

Ciclo de una ronda (ronda:)

Cada ronda atraviesa tres fases secuenciales:

- Espera de 3 segundos con los 4 LEDs encendidos (etiqueta espera_3s): se usa el mecanismo de polling sobre el contador de ciclos descrito en el Informe 1, comparando contra la constante TRES_S.
- Selección del LED objetivo (etiquetas tgt0–tgt3, tgt_ok): se calcula la semilla pseudoaleatoria a partir del contador de ciclos y el número de rondas jugadas, y se enciende únicamente el LED correspondiente (mecanismo descrito en el informe 1). En este punto se toma también la marca de tiempo inicial t0 (x9) que da comienzo a la medición del tiempo de reacción.
- Espera de la pulsación (etiquetas soltar, presion): primero se exige que no haya ningún botón presionado (actuando como anti-rebote y asegurando que la medición parta de un estado limpio), y luego se espera a que se presione alguno. Si el botón presionado coincide exactamente con la máscara del LED objetivo (x2), la ronda se considera acertada y el programa salta a acierto; cualquier otra combinación de botones se trata como error y el programa salta a error.

Bifurcación: acierto o error

En caso de acierto (acierto:), el programa mide el tiempo transcurrido desde t0, lo convierte a décimas de segundo, lo acumula en la suma total, lo muestra en los displays en formato decimal, incrementa el contador de rondas correctas, y vuelve a ronda: si aún no se completan las 10 rondas correctas.

En caso de error (error:), el programa muestra una indicación visual de error (“EE” en los displays, un patrón distinto en los LEDs) durante aproximadamente 0,5 segundos, y vuelve a ronda: sin modificar los contadores de rondas correctas ni la suma acumulada — es decir, la ronda se reinicia desde cero, pero el progreso ya logrado en rondas anteriores se conserva.

Finalización (fin:)

Cuando el contador de rondas correctas alcanza 10, el programa sale del ciclo de rondas, calcula el promedio de las décimas acumuladas (con redondeo), lo muestra en los displays,

y entra en un bucle infinito (fin: j fin) que deja ese valor visible de forma permanente — el programa no reinicia automáticamente ni acepta nuevas entradas una vez que se terminan las 10 rondas de juego.

A continuación, se detalla y justifica el código de cada una de las fases.

Fase de espera inicial (3 segundos)

Esta fase se documenta en el informe 1 mediante la perspectiva del hardware y las decisiones tomadas. Esta sección evalúa el funcionamiento lógico del código.

```
ronda:
   addi x5, x0, 15
   sw   x5, LEDS(x1)
   lw   x9, CICLOS(x1)      # t0 de la fase de espera
espera_3s:
   lw   x10, CICLOS(x1)
   sub  x5, x10, x9         # elapsed = t1 - t0 (resta módulo 2^32)
   bltu x5, x13, espera_3s  # aún no pasan los 3 s
```

Al entrar a cada ronda, el programa enciende los 4 LEDs (x5 = 15 = 0b1111) y toma inmediatamente la marca de tiempo t0 (x9) leyendo el contador de ciclos. A partir de ahí, el bucle espera_3s reutiliza exactamente el mismo patrón de medición que se usará luego para cronometrar la reacción del jugador: lectura de t1, cálculo de elapsed = t1 − t0 por resta, y comparación contra un umbral — en este caso, la constante x13 (TRES_S, 75.000.000 ciclos).

Dos detalles relevantes de implementación:

- La marca de tiempo t0 se toma una sola vez, justo antes del bucle. Esto es necesario para que elapsed mida correctamente el tiempo transcurrido desde el inicio de la espera, y no se reinicie en cada vuelta del bucle.
- La condición de salida usa bltu (branch less than, unsigned), es decir, una comparación sin signo. Esto es lo que hace válida la resta módulo 2^{32} frente al desborde del contador de ciclos, como se explicó en el Informe 1: si se hubiera usado una comparación con signo (blt), un valor de elapsed que por aritmética de complemento a dos se interprete como negativo (bit más significativo en 1) podría producir un resultado incorrecto cerca del punto de desborde.

Dado que esta fase es idéntica en cada una de las 10 rondas (se repite completa cada vez que el programa vuelve a la etiqueta ronda:), su costo en tiempo real del juego es fijo: cada ronda exige al jugador al menos 3 segundos de espera antes de que aparezca el LED objetivo, independientemente de cuántas rondas lleve jugadas.

Selección del LED objetivo

Esta sección también se apoya en la explicación conceptual ya dada en el Informe 1. Aquí se documenta el código completo, incluyendo el bloque de traducción del valor 0–3 a la

máscara one-hot que efectivamente se escribe en los LEDs.

```
lw   x5, CICLOS(x1)      # semilla
xor  x5, x5, x6          # semilla ^ rondas
add  x10, x5, x5         # x10 = 2*semilla
xor  x5, x5, x10         # semilla ^ 2*semilla (mezcla extra)
andi x5, x5, 3           # 0..3
# máscara one-hot SIN shifts: comparar/decrementar
beqz x5, tgt0
addi x5, x5, -1
beqz x5, tgt1
addi x5, x5, -1
beqz x5, tgt2
j    tgt3
tgt0:
   addi x2, x0, 1        # LED 0 -> 0001
   j    tgt_ok
tgt1:
   addi x2, x0, 2        # LED 1 -> 0010
   j    tgt_ok
tgt2:
   addi x2, x0, 4        # LED 2 -> 0100
   j    tgt_ok
tgt3:
   addi x2, x0, 8        # LED 3 -> 1000
tgt_ok:
   sw   x2, LEDS(x1)     # se enciende SOLO el LED objetivo
   lw   x9, CICLOS(x1)   # t0: la medición empieza con la ronda
```

Las primeras cinco instrucciones calculan la semilla mezclada y la reducen al rango 0–3, como se explicó en el Informe 1. El bloque siguiente resuelve, sin usar shifts, el problema de convertir ese número de 2 bits en una máscara one-hot de 4 bits:

- Se compara x5 contra 0 (beqz x5, tgt0). Si es 0, el LED objetivo es el 0.
- Si no, se decrementa x5 en 1 y se repite la comparación contra 0 para el caso 1 (tgt1).
- El mismo patrón se repite para el caso 2 (tgt2).
- Si ninguna de las tres comparaciones anteriores tuvo éxito, el valor original solo puede haber sido 3, por lo que el programa salta directamente a tgt3 sin necesidad de una cuarta comparación.

Cada etiqueta tgtN carga en x2 la constante one-hot correspondiente usando addi con x0 como fuente (equivalente a cargar un literal), y todas convergen en tgt_ok, donde se escribe x2 en el registro de LEDs — encendiendo únicamente el LED objetivo, y apagando los otros tres que estaban encendidos desde la fase de espera — y se toma la marca de tiempo t0 (x9) que dará inicio a la medición del tiempo de reacción del jugador.

Costo en instrucciones de este enfoque

Este método de traducción por comparación/decremento ejecuta, en el peor caso (LED 3), tres comparaciones beqz y dos decrementos addi antes de llegar al salto final — un costo mayor que el de un único shift de bits, pero necesario dado que el hardware no lo soporta. Al ser un bloque que se ejecuta una sola vez por ronda (no dentro de un bucle de espera), este costo adicional de instrucciones no tiene impacto perceptible en el tiempo de juego.

Medición del tiempo de respuesta

Antes de poder medir el tiempo de reacción, el programa debe asegurarse de que no haya ningún botón presionado al momento de encender el LED objetivo (por ejemplo, si el jugador todavía tenía presionado el botón de la ronda anterior). Esto se resuelve con dos bucles consecutivos:

```
soltar:
   lw   x3, BOTONES(x1)
   bnez x3, soltar          # espera a que no haya botón apretado
presion:
   lw   x3, BOTONES(x1)
   beqz x3, presion         # nada presionado: seguir esperando
   beq  x3, x2, acierto     # botón correcto -> termina la ronda
   j    error               # cualquier otra pulsación -> error
```

El bucle soltar actúa como anti-rebote y como garantía de que la medición de tiempo comienza desde un estado limpio: mientras el registro de botones no sea 0, el programa espera. Una vez que no hay botones presionados, el bucle presion comienza a sondear activamente hasta detectar una pulsación. Si el valor leído coincide exactamente con la máscara del LED objetivo (x2), la ronda se da por acertada; cualquier otro valor distinto de 0 —ya sea un botón incorrecto o una combinación de varios botones— se trata como error.

Cálculo del tiempo transcurrido (elapsed)

Al detectar el acierto, el programa calcula el tiempo transcurrido desde que se encendió el LED objetivo:

```
acierto:
   lw   x10, CICLOS(x1)     # t1
   sub  x10, x10, x9        # elapsed en ciclos
```

Esto reutiliza el mismo patrón t0/t1/resta ya usado en la fase de espera de 3 segundos: t0 (x9) se tomó justo después de encender el LED objetivo, y t1 (x10) se toma en el instante en que se detecta la pulsación correcta. La resta entrega el número de ciclos de reloj que el jugador tardó en reaccionar, y —al igual que en la espera de 3 segundos— el resultado es correcto aun si el contador de ciclos se hubiera desbordado en el intervalo, gracias a la aritmética módulo 2^{32}.

Conversión de ciclos a décimas de segundo

El valor en ciclos no es directamente utilizable para mostrarlo en los displays; debe convertirse a décimas de segundo. Como la ALU no soporta división, esto se resuelve por resta repetida de la constante TENTH (x12, 2.500.000 ciclos):

```
addi x8, x0, 0          # décimas = 0
decimas:
   bltu x10, x12, dec_ok    # elapsed < 1 décima -> listo
   sub  x10, x10, x12       # elapsed -= TENTH  (división por resta)
   addi x8, x8, 1           # décimas++
   j    decimas
dec_ok:
   add  x7, x7, x8          # suma total += décimas de la ronda
```

El bucle decimas implementa división entera por resta repetida: mientras el tiempo transcurrido (x10) sea mayor o igual a una décima de segundo (TENTH), se le resta una décima y se incrementa el contador de décimas (x8). Cuando elapsed queda por debajo de TENTH, el cociente acumulado en x8 es exactamente el número de décimas de segundo (redondeado hacia abajo, truncado) que tardó el jugador en reaccionar. Finalmente, ese valor se suma al acumulador total de décimas de las 10 rondas (x7), que se usará más adelante para calcular el promedio final.

Costo de este enfoque

El número de iteraciones de este bucle depende directamente del tiempo de reacción del jugador: para un reflejo típico de 2 a 4 décimas de segundo, el bucle ejecuta solo 2 a 4 iteraciones, lo cual es perfectamente aceptable en términos de tiempo de ejecución del programa (del orden de decenas de ciclos de CPU, insignificante frente a los 2.500.000 ciclos que representa cada década medida).

Conversión a BCD y despliegue en displays

Una vez calculadas las décimas de segundo de la ronda (x8), el programa debe mostrar ese valor en los dos displays de 7 segmentos, en formato decimal (por ejemplo, “23” debe leerse como 2 décadas y 3 unidades — es decir, dos dígitos decimales, no un valor hexadecimal). El código correspondiente es:

```
mv   x5, x8              # unidades candidatas
addi x11, x0, 0          # decenas = 0
bcd_ronda:
   bltu x5, x14, bcd_r_fin  # x5 < 10 -> ya están las cifras
   sub  x5, x5, x14         # división por 10 por resta repetida
   addi x11, x11, 1
   j    bcd_ronda
bcd_r_fin:
   add  x4, x11, x11        # 2*decenas
   add  x4, x4, x4          # 4*decenas
```

```
  add  x4, x4, x4         # 8*decenas
  add  x4, x4, x4         # 16*decenas
  or   x4, x4, x5         # + unidades
  sw   x4, DIGITOS(x1)    # displays muestran "dd" en decimal
```

Separación en decenas y unidades

El primer bucle (bcd_ronda) descompone el valor de décimas (x8) en decenas (x11) y unidades (x5) mediante resta repetida por 10, el mismo patrón de división entera usado para convertir ciclos a décimas: mientras el valor restante sea mayor o igual a 10 (constante DIEZ, x14), se le resta 10 y se incrementa el contador de decenas. Al salir del bucle, x11 contiene las decenas y x5 contiene las unidades (ambas en el rango 0–9, dado que el promedio y las rondas individuales no superan los 99).

Armado del valor BCD de 2 dígitos

El controlador de displays (pochoco_periph) espera un único byte en el que el nibble alto corresponde al primer dígito (decenas) y el nibble bajo al segundo dígito (unidades). Como la ALU no soporta shifts, el nibble alto no puede construirse desplazando las decenas 4 bits a la izquierda; en su lugar, se obtiene multiplicando las decenas por 16 mediante cuatro sumas dobles consecutivas (2×, 4×, 8×, 16×), equivalentes a un shift de 4 bits pero implementadas con la única operación aritmética disponible. El resultado se combina con las unidades mediante un or (válido porque las unidades ocupan solo los 4 bits bajos y nunca se superponen con el nibble alto recién calculado), y el byte final se escribe directamente en el offset DIGITOS.

Por qué el resultado se interpreta como decimal y no como hexadecimal

Este es el punto clave que satisface el requisito del enunciado de mostrar el tiempo “en decimal, no hex”: aunque el valor que llega al hardware es, en términos de representación binaria, un byte interpretado por el decodificador de 7 segmentos como dos dígitos hexadecimales, el programa construye ese byte de forma que cada nibble nunca excede 9 (porque decenas y unidades provienen de una división por 10, no por 16). Esto hace que el decodificador hexadecimal muestre, por construcción, solamente dígitos del 0 al 9 en cada display — logrando una lectura decimal correcta sin necesidad de un decodificador BCD dedicado en el hardware. Es, en ese sentido, una codificación BCD (binary-coded decimal) empaquetada en un byte, reutilizando el decodificador hexadecimal ya existente en el periférico.

Manejo de pulsaciones incorrectas

Cuando el jugador presiona un botón distinto del LED objetivo (o varios botones a la vez), el programa salta a la etiqueta error:, encargada de dar una señal visual clara del fallo y de reiniciar la ronda sin afectar el progreso ya acumulado:

```
error:
```

```
    li   x5, 0xEE
    sw   x5, DIGITOS(x1)
    addi x5, x0, 5
    sw   x5, LEDS(x1)
    lw   x9, CICLOS(x1)
    li   x15, 12500000      # 0,5 s en ciclos
err_espera:
    lw   x10, CICLOS(x1)
    sub  x5, x10, x9
    bltu x5, x15, err_espera
    j    ronda              # reinicia la ronda actual;
                            # x6 y x7 NO se tocan (acumulan)
```

Indicación visual del error

El programa escribe el valor 0xEE directamente en los displays. A diferencia de los valores calculados por conversión a BCD (donde ambos nibbles siempre quedan entre 0 y 9, ver sección anterior), aquí se usa deliberadamente un valor fuera de ese rango: el decodificador hexadecimal de 7 segmentos (hex2seg, Informe 1) interpreta cada nibble E como la letra “E”, mostrando el mensaje “EE” en los dos displays, una señal inequívoca de error para el jugador. Simultáneamente, se escribe el valor 5 (0b0101) en el registro de LEDs, encendiendo un patrón distinto al usado durante el juego normal (LEDs todos encendidos en la espera, o un único LED objetivo durante la ronda), reforzando visualmente que algo salió mal.

Duración fija del aviso de error

El aviso se mantiene visible durante aproximadamente 0,5 segundos, usando una vez más el mismo patrón de medición por diferencia de ciclos (t0/t1/resta) ya documentado en las secciones anteriores, pero con un umbral distinto: la constante 12.500.000 ciclos, que corresponde a medio segundo a 25 MHz (0,5 s × 25.000.000 Hz = 12.500.000 ciclos — la mitad exacta del valor de TENTH multiplicado por 5, consistente con la unidad de décima de segundo usada en el resto del programa). A diferencia de TENTH y TRES_S, este umbral no se define como constante .equ reutilizable sino que se carga directamente con li x15, 12500000 en el momento en que se necesita, ya que es un valor usado una única vez en todo el programa.

Reinicio de la ronda sin pérdida de progreso

Transcurrido el medio segundo, el programa salta de vuelta a la etiqueta ronda:, repitiendo desde cero las tres fases de una ronda (espera de 3 segundos, selección de LED objetivo, espera de pulsación). El punto central de este bloque, explicitado incluso en el propio comentario del código fuente, es que los registros x6 (rondas correctas acumuladas) y x7 (suma de décimas acumuladas) no se tocan en ningún momento del manejo de error: el jugador pierde únicamente el intento fallido, no el progreso de las rondas ya acertadas anteriormente. Esto es coherente con el requisito del enunciado de que “las rondas

correctas acumuladas se conservan” tras un error.

Acumulación y cálculo del promedio final

Acumulación durante las 10 rondas

Como se adelantó en secciones anteriores, cada vez que el jugador acierta una ronda, su tiempo de reacción en décimas de segundo (x8) se suma al acumulador total x7 (add x7, x7, x8, dentro del bloque dec_ok), y el contador de rondas correctas x6 se incrementa en 1. El programa compara x6 contra la constante DIEZ (x14) para decidir si continúa jugando o pasa al cálculo final:

```
addi x6, x6, 1          # rondas correctas++
bne  x6, x14, ronda     # menos de 10 -> otra ronda
```

Mientras x6 sea distinto de 10, el programa vuelve a ronda: y repite el ciclo completo (espera de 3 s, selección de LED, medición). Cuando x6 llega exactamente a 10, el programa continúa en la siguiente instrucción, entrando al bloque de cálculo del promedio.

División por 10 del acumulado

```
mv   x5, x7             # promedio = suma / 10 por resta
addi x11, x0, 0         # cociente
div10:
   bltu x5, x14, div_fin
   sub  x5, x5, x14
   addi x11, x11, 1
   j    div10
div_fin:
```

Al terminar las 10 rondas, x7 contiene la suma total de décimas de segundo de todas las rondas correctas. El promedio se calcula dividiendo esa suma por 10, nuevamente mediante resta repetida: mientras el resto (x5) sea mayor o igual a 10, se le resta 10 y se incrementa el cociente (x11). Al salir del bucle, x11 contiene el cociente entero (la parte entera del promedio) y x5 contiene el resto de la división (el residuo necesario para decidir el redondeo).

Criterio de redondeo

```
li   x15, 5
bltu x5, x15, sin_red     # resto < 5: no redondea
addi x11, x11, 1          # redondeo: resto >= 5 -> +1 décima
sin_red:
```

El programa aplica redondeo al entero más cercano, comparando el resto de la división (x5) contra 5: si el resto es menor que 5, el cociente x11 queda tal cual (redondeo hacia abajo); si el resto es mayor o igual a 5, se incrementa el cociente en 1 (redondeo hacia arriba). Este es el criterio de redondeo estándar “mitad hacia arriba” (round-half-up) aplicado sobre un cociente entero con resto, equivalente a redondear el promedio real (con decimales) a la

décima de segundo entera más cercana.

Conversión a BCD y despliegue del promedio

```
mv   x5, x11              # a BCD
addi x11, x0, 0
bcd_prom:
   bltu x5, x14, bcd_p_fin
   sub  x5, x5, x14
   addi x11, x11, 1
   j    bcd_prom
bcd_p_fin:
   add  x4, x11, x11        # 16*decenas (4 dobles, sin shifts)
   add  x4, x4, x4
   add  x4, x4, x4
   add  x4, x4, x4
   or   x4, x4, x5
   sw   x4, DIGITOS(x1)      # displays muestran el promedio
fin:
   j    fin                  # queda mostrando el promedio
```

El valor final (ya redondeado) se convierte a BCD de 2 dígitos exactamente con el mismo procedimiento usado para mostrar cada ronda individual: separación en decenas/unidades por resta repetida, y armado del byte de displays multiplicando las decenas por 16 mediante cuatro sumas dobles. El resultado se escribe en los displays, y el programa entra en un bucle infinito (fin: j fin) que detiene la ejecución dejando el promedio visible de forma indefinida — el juego no se reinicia automáticamente ni acepta más entradas una vez mostrado el resultado final.

Ejemplo numérico ilustrativo

Para verificar la lógica descrita en las secciones anteriores, se presenta a continuación un ejemplo numérico completo, calculado a mano, que recorre una ronda individual y el cálculo del promedio final.

Ejemplo de una ronda

Supongamos que, en una ronda dada, el jugador reacciona en 2,35 segundos reales. A 25 MHz, esto corresponde a un número de ciclos:

elapsed = 2,35 s × 25.000.000 Hz = 58.750.000 ciclos

Este es el valor que el programa obtendría como t1 − t0 (sección “Medición del tiempo de respuesta”). El bucle decimas resta repetidamente TENTH (2.500.000 ciclos) hasta que el resto cae por debajo de ese umbral:

58.750.000 ÷ 2.500.000 = 23 restas completas, con resto 1.250.000

Es decir, el bucle itera 23 veces, x8 queda en 23 (23 décimas de segundo = 2,3 s), y el resto final (1.250.000 ciclos = 0,125 s) se descarta — la conversión trunca hacia abajo, no redondea, por lo que 2,35 s reales se registran como 2,3 s. Este valor (23) se suma al acumulador total x7.

Conversión a BCD de esta ronda

Con x8 = 23, el bucle bcd_ronda resta 10 dos veces (23 − 10 = 13, 13 − 10 = 3), dejando x11 = 2 (decenas) y x5 = 3 (unidades). El byte de displays se arma como:

16 × 2 (decenas) = 32 = 0x20

0x20 OR 3 (unidades) = 0x23

El valor 0x23 escrito en DIGITOS hace que el decodificador hexadecimal muestre “2” en el primer display y “3” en el segundo — es decir, “23”.

Ejemplo de cálculo del promedio

Supongamos que, al cabo de las 10 rondas correctas, la suma acumulada en x7 es:

x7 = 23 + 18 + 31 + 15 + 27 + 19 + 22 + 16 + 20 + 24 = 215 (décimas de segundo)

El bucle div10 resta 10 repetidamente: 215 ÷ 10 = 21 restas completas, con resto 5. Es decir, x11 = 21 (cociente) y x5 = 5 (resto).

Como el resto (5) es mayor o igual a 5, se aplica el redondeo hacia arriba: x11 pasa de 21 a 22. El promedio final mostrado es, entonces, 22 décimas de segundo = 2,2 segundos — frente a un promedio real de 215/10 = 21,5 décimas, correctamente redondeado al entero más cercano.

Conversión a BCD del promedio

Con x11 = 22 décimas, el bucle bcd_prom resta 10 dos veces (22 − 10 = 12, 12 − 10 = 2), dejando decenas = 2 y unidades = 2. El byte final es:

16 × 2 (decenas) = 32 = 0x20

0x20 OR 2 (unidades) = 0x22

El valor 0x22 se escribe en DIGITOS, mostrando “22” en los displays — el promedio final de 2,2 segundos, en formato decimal.

Conclusión

El programa game.s implementa de forma completa la lógica del juego de reflejos exigida por el enunciado, apoyándose consistentemente en el contador de ciclos documentado en el Informe 1: cada fase de temporización (espera de 3 segundos, medición del tiempo de reacción, duración del aviso de error) reutiliza el mismo patrón de medición por diferencia de lecturas (t0, t1, resta módulo 2^{32}), lo que da uniformidad y coherencia al diseño del software.

Las limitaciones de la ALU se resolvieron sistemáticamente con sumas y restas repetidas: división entera por resta repetida (conversión de ciclos a décimas, separación en decenas/unidades, promedio final), y multiplicación por 16 mediante cuatro sumas dobles (armado del byte de displays). Este patrón se repite en al menos tres puntos distintos del programa, lo que demuestra que la solución adoptada es reutilizable y no un parche puntual.

El programa cumple además los requisitos funcionales específicos del enunciado: muestra el tiempo en formato decimal (no hexadecimal) explotando de forma controlada el decodificador hexadecimal existente en el hardware; acumula el progreso de las 10 rondas correctas de forma independiente de los errores cometidos; aplica redondeo estándar al calcular el promedio final; y distingue visualmente los tres estados del juego (espera, ronda activa, error) mediante distintos patrones de LEDs y displays. El ejemplo numérico desarrollado en la sección anterior confirma que la cadena completa de cálculos —desde ciclos de reloj hasta el valor final mostrado en pantalla— es matemáticamente consistente y reproducible a mano.